

Model-Driven Test Design

Clase 2:Fault & Failure Model (RIPR)| Requisitos y criterios de test|Tipos actividades de Testing

Valeria Bengolea

Estas transparencias están basadas en las desarrolladas por Ammann & Offutt como acompañamiento de su libro Introduction to Software Testing (2nd Edition) y por Manuel Núñez

Complejidad del testing de Software

Ningún otro campo de la ingeniería construye productos tan complicados como el software.

De hecho, el término **corrección** no tiene significado en otros contextos.

¿Es un edificio correcto?

¿Es un coche correcto?

¿Es la red de metro correcta?

Es importante **abstraer** para **reducir** la **complejidad**.

Este es el propósito principal del diseño de tests basado en modelos.

El *modelo* es una estructura abstracta.

Testing & Debugging

Testing: Evaluar software observando su ejecución.

Test Failure: Ejecución de un test que da lugar a un fallo en el software.

Debugging: El proceso de encontrar un defecto (fault) a la vista de un fallo (failure).

Testing & Debugging

Testing: Evaluar software observando su ejecución.

Test Failure: Ejecución de un test que da lugar a un fallo en el software.

Debugging: El proceso de encontrar un defecto (fault) a la vista de un fallo (failure).

No todos los inputs darán lugar a que un defecto
“provoque” un fallo.

Fault & Failure Model (RIPR)

Se deben dar cuatro condiciones para **observar** un fallo.

- ✓ **_Reachability**: Lugar (o lugares) del programa que contienen el defecto (fault) que debe alcanzarse.
- ✓ **Infection**: El estado del programa debe ser incorrecto.
- ✓ **Propagation**: El estado infectado debe causar que algún output o estado final del programa sea incorrecto.
- ✓ **Revealability**: El tester debe observar (parte de) la porción incorrecta del estado del programa

Modelo RIPR

Modelo RIPR

- **Reachability**
- **Infection**
- **Propagation**
- **Revealability**

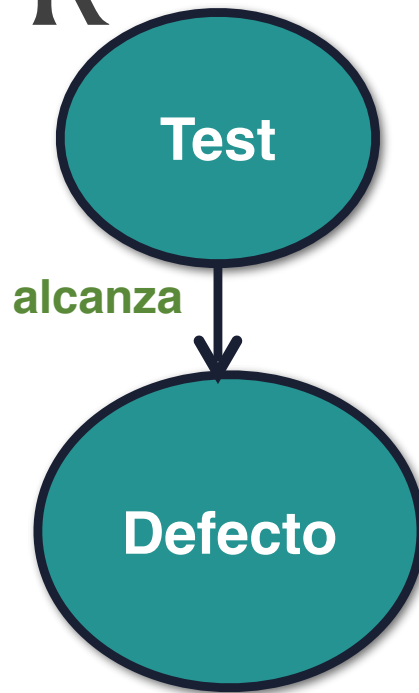
Modelo RIPR



Test

- **Reachability**
- **Infection**
- **Propagation**
- **Revealability**

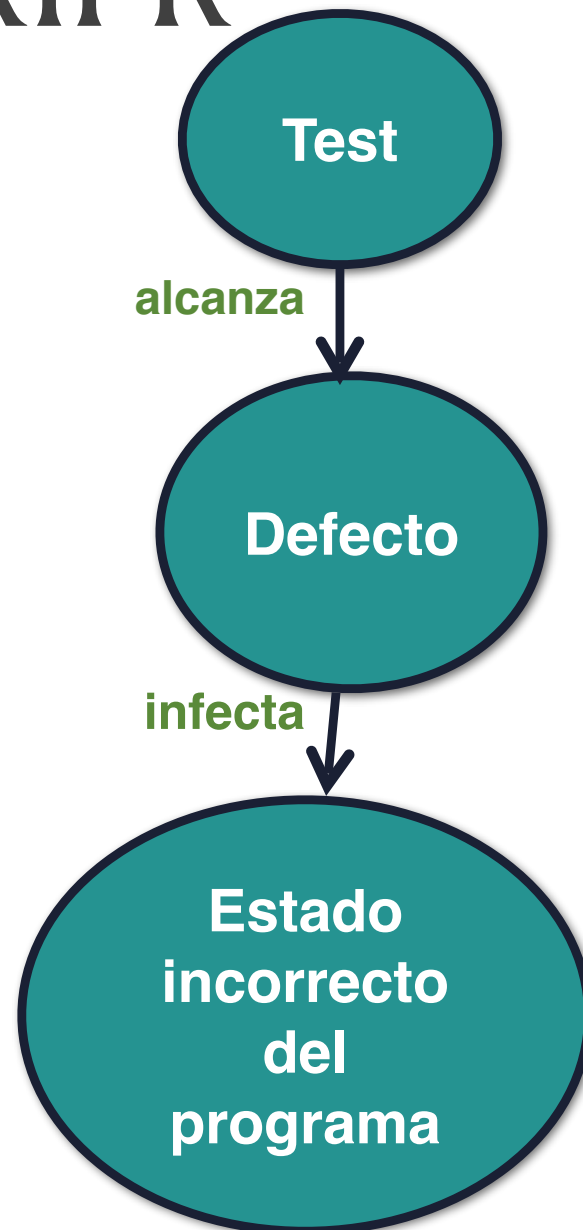
Modelo RIPR



- **Reachability**
- **Infection**
- **Propagation**
- **Revealability**

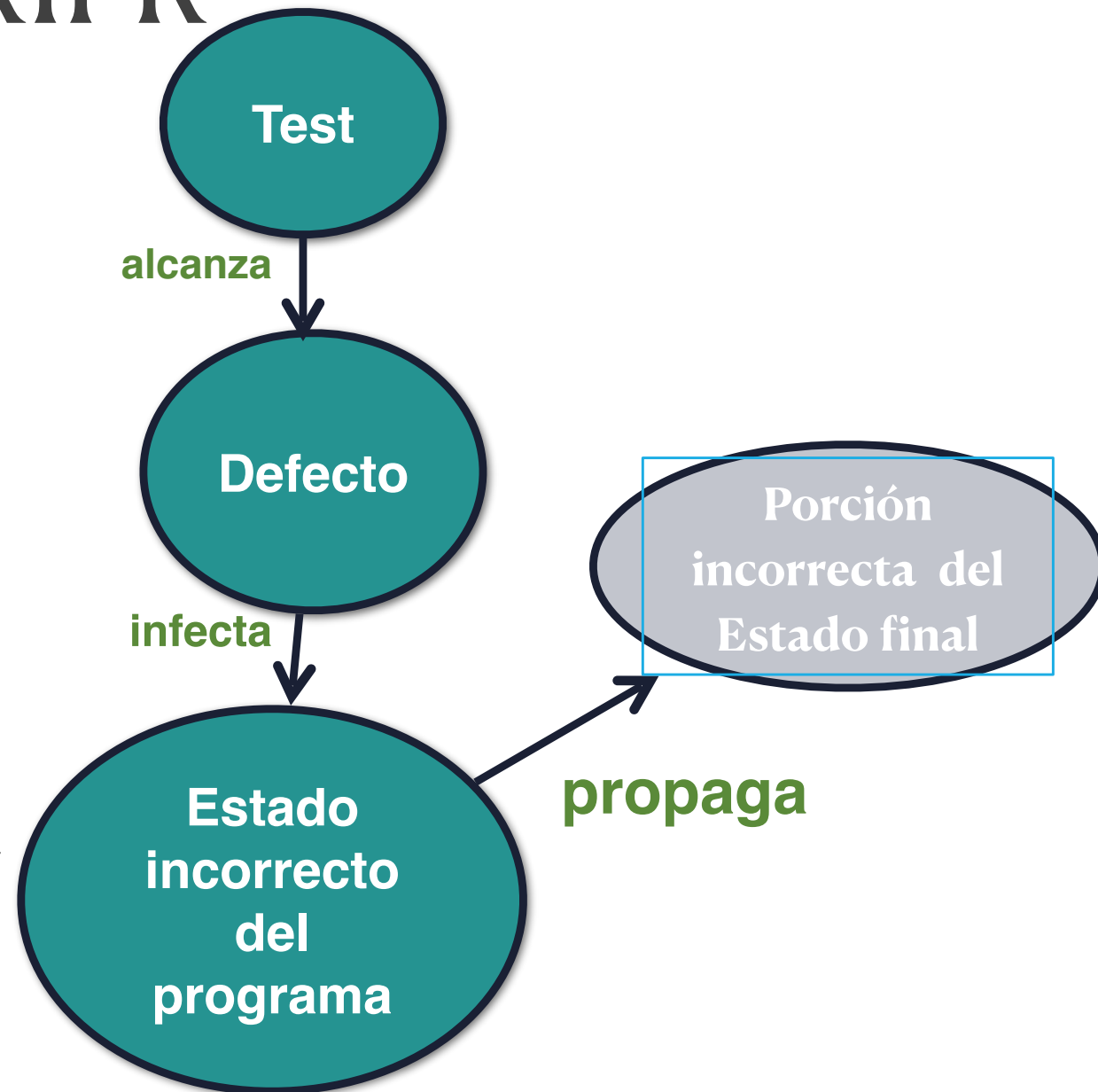
Modelo RIPR

- **Reachability**
- **Infection**
- **Propagation**
- **Revealability**



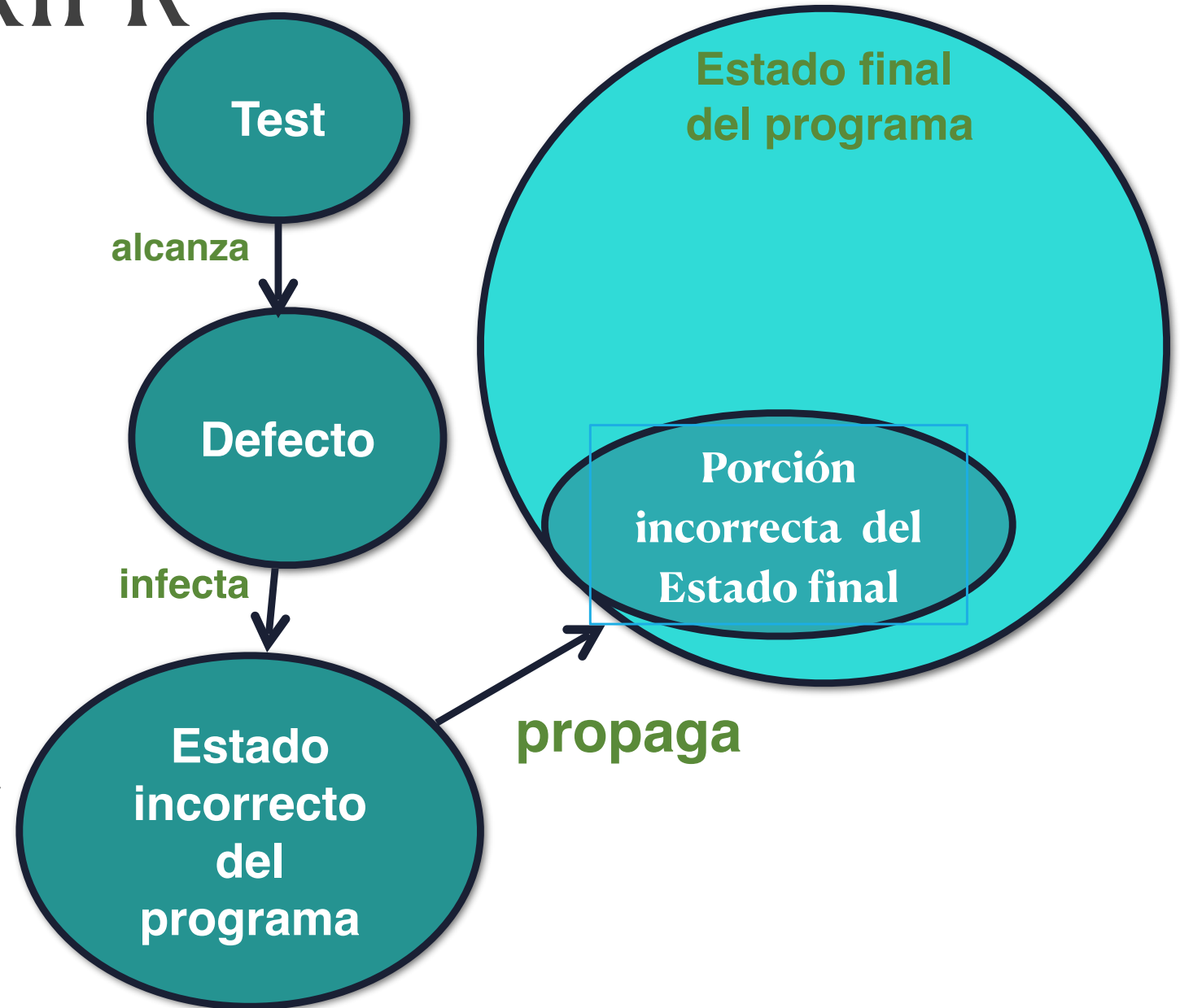
Modelo RIPR

- **Reachability**
- **Infection**
- **Propagation**
- **Revealability**



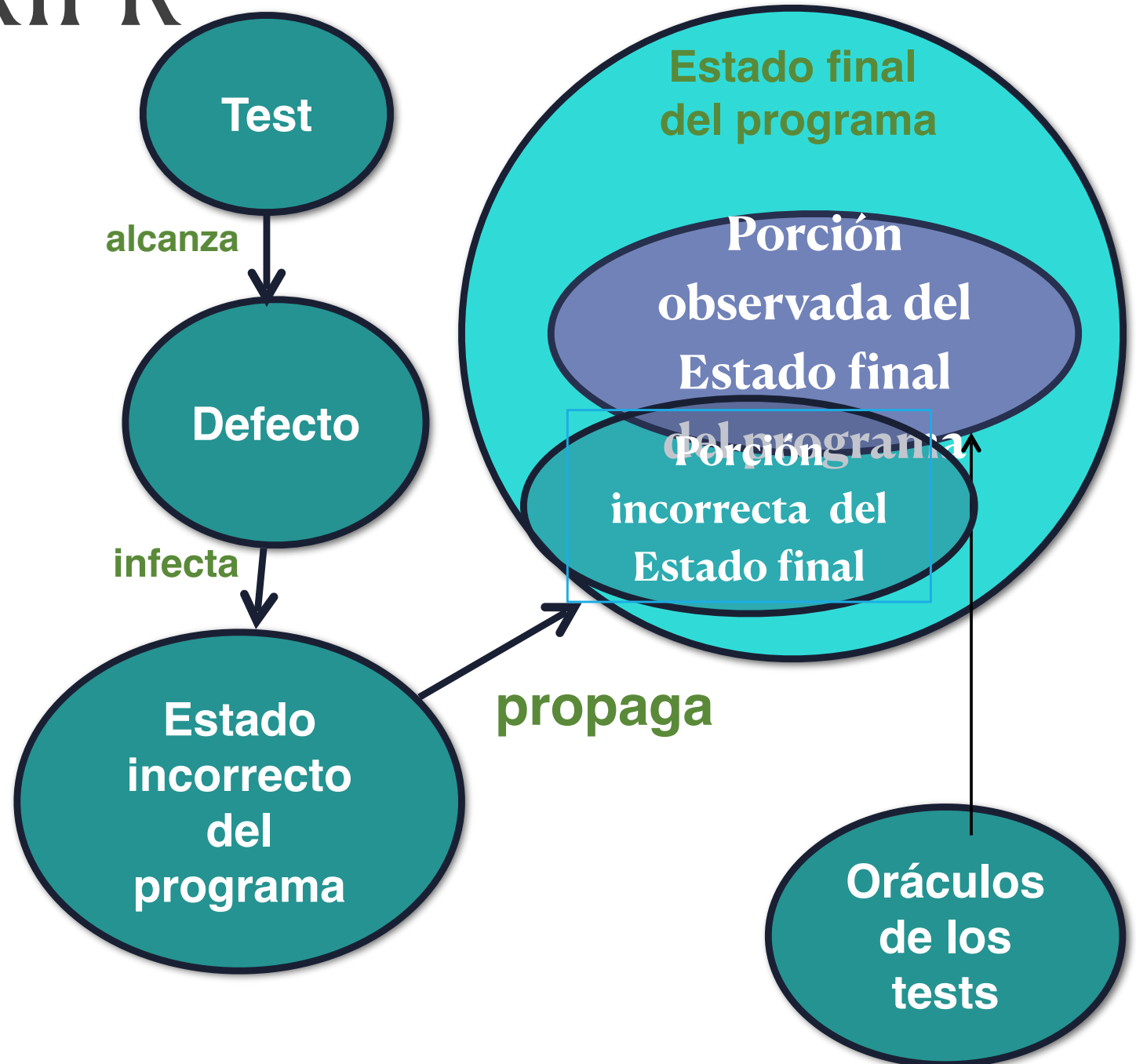
Modelo RIPR

- **Reachability**
- **Infection**
- **Propagation**
- **Revealability**



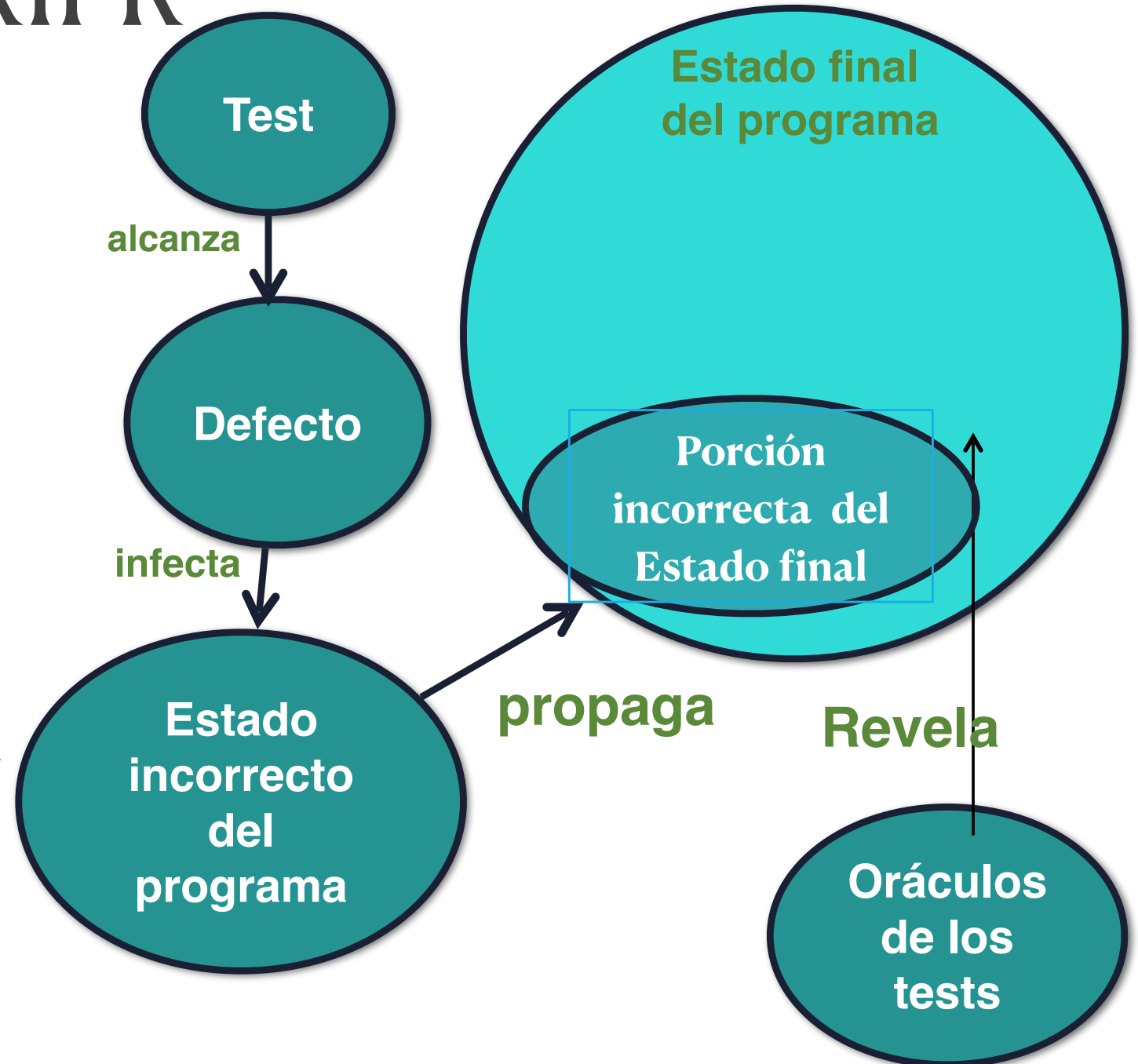
Modelo RIPR

- **Reachability**
- **Infection**
- **Propagation**
- **Revealability**



Modelo RIPR

- **Reachability**
- **Infection**
- **Propagation**
- **Revealability**



Criterios de cobertura

Incluso los programas más pequeños tienen demasiados inputs para testearlos completamente.

Criterios de cobertura

Incluso los programas más pequeños tienen demasiados inputs para testearlos completamente.

```
private static double computeAverage (int A, int B, int C)
```


Criterios de cobertura

Incluso los programas más pequeños tienen demasiados inputs para testearlos completamente.

4.000.000.000 valores posibles.

(32 bits)



```
private static double computeAverage (int A, int B, int C)
```

Criterios de cobertura

Incluso los programas más pequeños tienen demasiados inputs para testearlos completamente.

4.000.000.000 valores posibles.

(32 bits)



```
private static double computeAverage (int A, int B, int C)
```

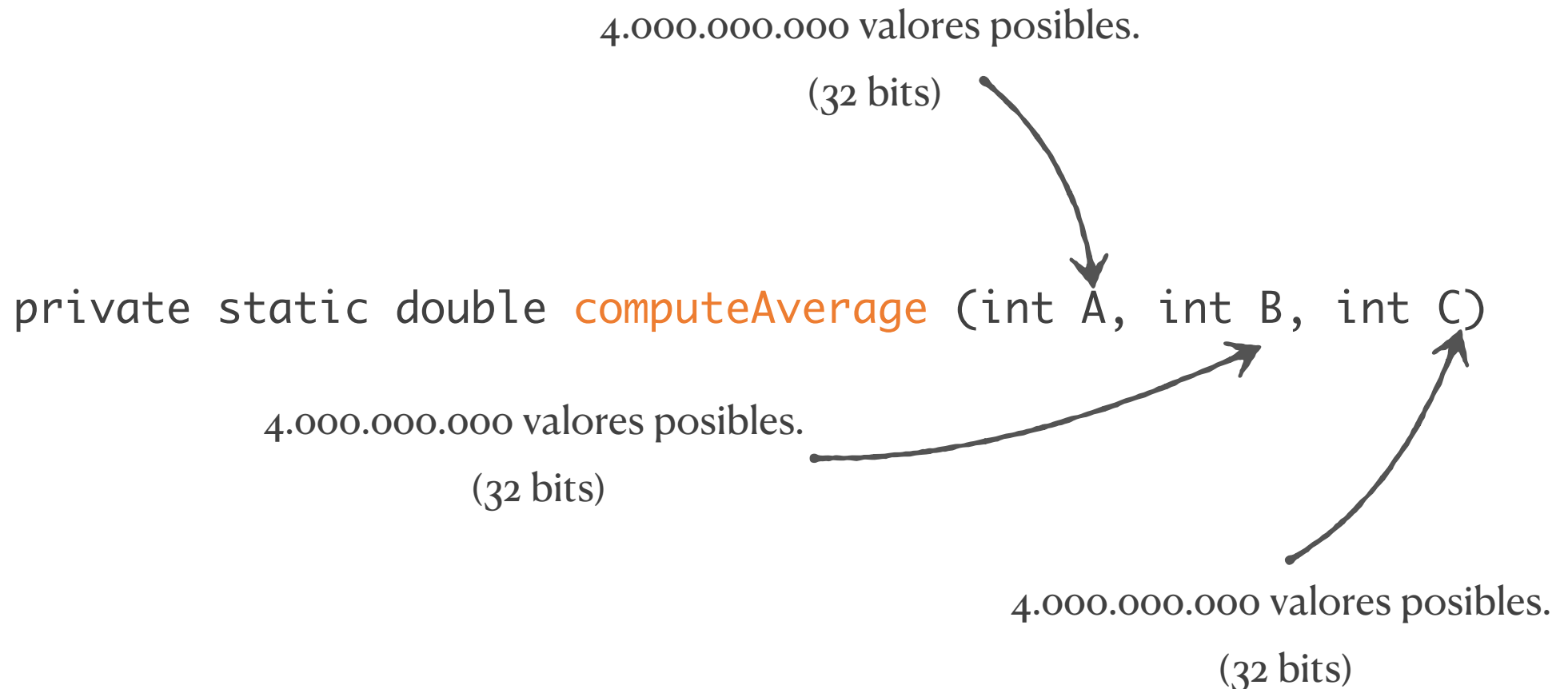
4.000.000.000 valores posibles.

(32 bits)



Criterios de cobertura

Incluso los programas más pequeños tienen demasiados inputs para testearlos completamente.



Criterios de cobertura

Incluso los programas más pequeños tienen demasiados inputs para testearlos completamente.

Tenemos alrededor de
80.000.000.000.000.000.000.000.000.000 de
tests posibles!!

private static double **computeAverage** (int A, int B, int C)

4.000.000.000 valores posibles.
(32 bits)

4.000.000.000 valores posibles.
(32 bits)

Criterios de cobertura

Incluso los programas más pequeños tienen demasiados inputs para testearlos completamente.

Tenemos alrededor de
80.000.000.000.000.000.000.000.000.000 de
tests posibles!!

```
private static double computeAverage (int A, int B, int C)
```

4.000.000.000 valores posibles.
(32 bits)

4.000.000.000 valores posibles.
(32 bits)

Esencialmente, podríamos asumir
que el espacio de inputs es infinito.

Criterios de cobertura

Los testers tienen que trabajar con espacios de inputs inmensos con el **objetivo** de encontrar el **menor número de inputs** que detecte la **mayoría de los problemas**.

Esta es la causa de dos problemas en testing:

1. Cómo buscamos en ese espacio de inputs
2. Cuando paramos de buscar?

Criterios de cobertura

Los criterios de cobertura nos dotan de maneras estructuradas y prácticas de buscar en el espacio de inputs de forma que

- ★ La búsqueda se realiza de manera minuciosa en ese espacio.
- ★ No se produzca demasiado solapamiento entre los tests.

Requisitos y criterios de test

Criterio de test: Colección de reglas y un proceso que define los requisitos de test. Por ejemplo,

- Cubrir cada línea de código.

- Cubrir cada requisito funcional.

Requisitos de test: Elemento específico de un artefacto de software que un caso de test debe satisfacer o cubrir. Por ejemplo,

- Cada línea de código podría dar lugar a un requisito.

- Cada requisito funcional podría dar lugar a un requisito.

Requisitos y criterios de test

Se han definido muchos criterios sobre diferentes artefactos de Software,
Si se abstraen estos artefactos a modelos matemáticos, muchos criterios terminan siendo los mismos!

Requisitos y criterios de test

Los criterios pueden reducirse a algunos pocos sobre cuatro tipos de estructuras matemáticas:

1. Dominio de Entradas
2. Grafos
3. Expresiones lógicas
4. Descripciones Sintácticas

Requisitos y criterios de test

Los criterios pueden reducirse a algunos pocos sobre cuatro tipos de estructuras matemáticas:

1. Dominio de Entradas
2. Grafos
3. Expresiones lógicas
4. Descripciones Sintácticas

Debido a que el proceso de Diseño de tests está basado en estos 4 modelos abstractos, se lo denomina **Model-Driven Test Design Process (MDTD)**

Caja blanca vs. caja negra

Testing de caja negra: Se generan tests a partir de descripciones externas del software que pueden ser especificaciones, requisitos u otros mecanismos de diseño.

Testing de caja blanca: Se generan tests a partir del código del software, específicamente, a partir de sus *ramas*, condiciones y líneas de código.

MDTD (Model-Driven Test Design)

- ★ MDTD (Model-Driven Test Design) hace que estas distinciones sean menos importantes.
- ★ La pregunta pasa a ser: ¿A partir de qué nivel de abstracción generamos los tests?

Model-Driven Test Design

El **diseño de tests** es el proceso consistente en diseñar valores de los inputs que testean de forma efectiva el software.

Model-Driven Test Design

El **diseño de tests** es el proceso consistente en diseñar valores de los inputs que testean de forma efectiva el software.

El diseño de tests es solo una de las diferentes actividades que constituyen el testing de software. Tiene dos características principales:

- Tiene una base más rigurosa (formalismos con base matemática).

- Consiste en crear modelos abstractos y manipularlos para diseñar test de calidad

Tipos de actividades en testing

Testing se puede dividir en los siguientes cuatro grandes grupos de actividades:

1. Diseño de tests.
2. Automatización de tests.
3. Ejecución de tests.
4. Evaluación de tests.



1.a) Basado en criterios.
1.b) Basados en factor humano.

Cada tipo de actividad requiere diferentes habilidades, conocimientos previos y formación.

Diseño de tests

Proceso de diseñar valores de entrada para testear el software efectivamente:

- ★ **Basado en criterios:** valores de los tests para satisfacer criterios de cobertura u otro objetivo.
- ★ **Basado en conocimiento humano:** Diseñar valores de los tests basándose en el conocimiento del dominio del programa y el conocimiento de testing del tester.

Otras actividades en testing

- ★ **Automatización de los test:** codificar los test en scripts ejecutables.
- ★ **Ejecución de los tests:** Correr los tests sobre el software y registrar los resultados
- ★ **Evaluación de los tests:** Evaluar los resultados y reportarlos a los desarrolladores/managers.

Otras actividades

- ★ **Gestión de tests:** Marcar líneas generales, organización del equipo, elección de criterios, grado de automatización.
- ★ **Mantenimiento de tests:** Guarda los tests para reutilizarlos en posteriores evoluciones del software.
- ★ **Documentación de tests:** Cada test debe estar bien documentado: criterios y requisitos de test satisfechos o justificación de tests diseñados a mano. Mantener documentación sobre los tests automatizados.

Model-Driven Test Diseño

Model-Driven Test Diseño

**Artefacto
software**

Model-Driven Test Diseño

DISEÑO

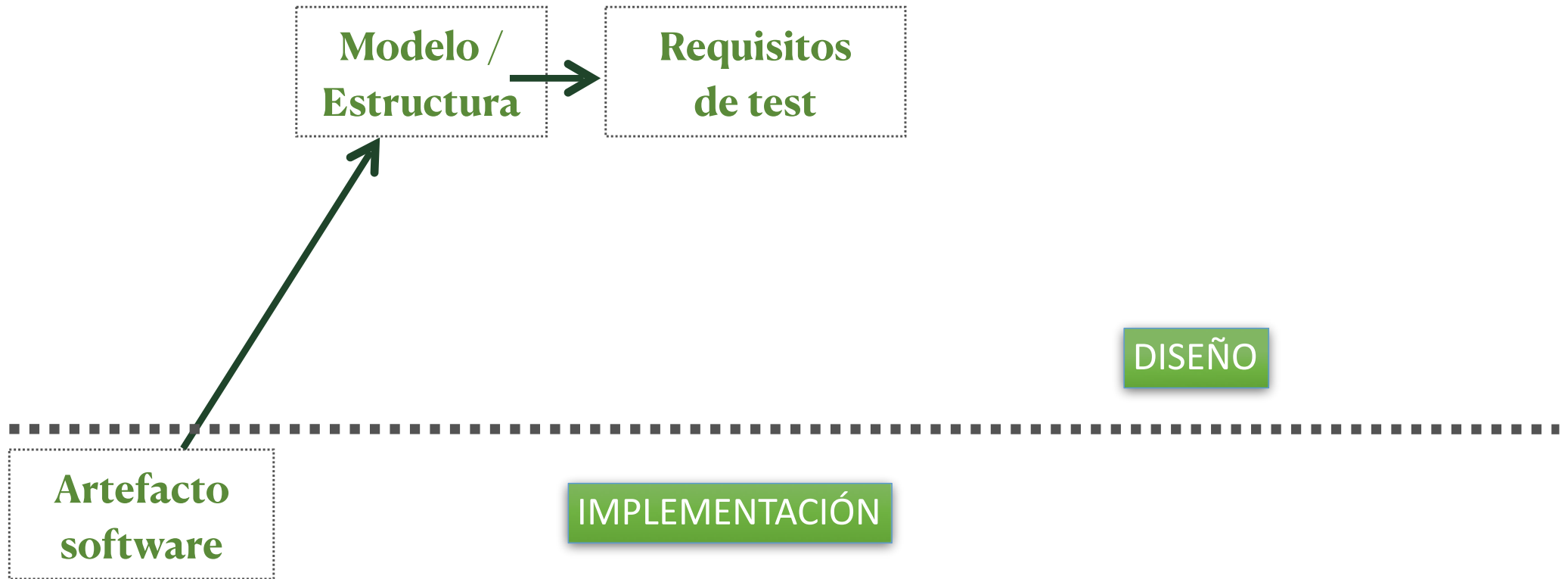
Artefacto
software

IMPLEMENTACIÓN

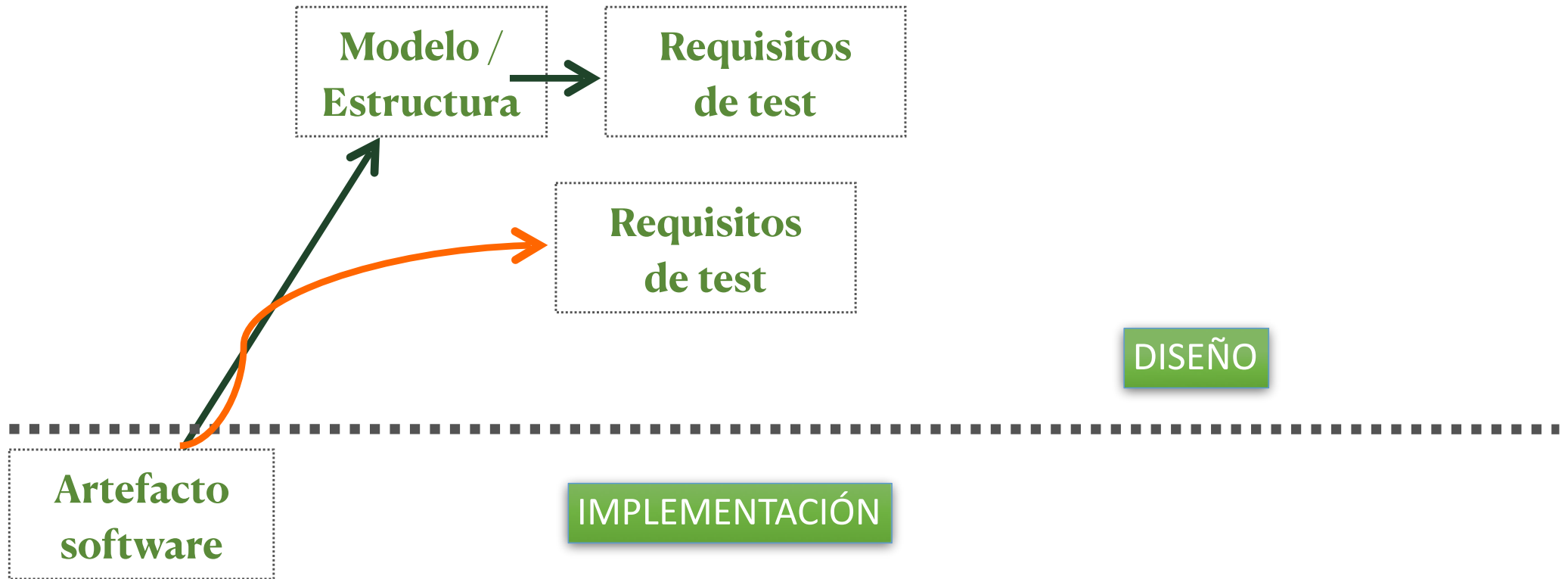
Model-Driven Test Diseño



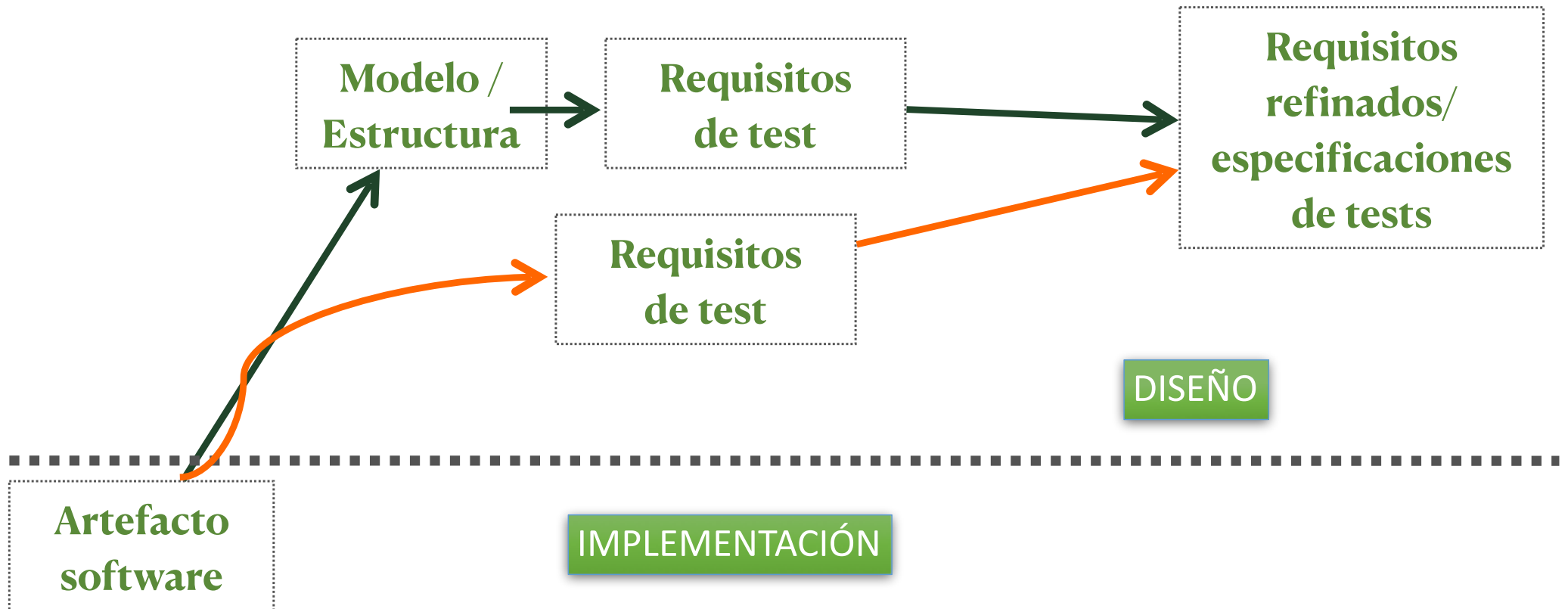
Model-Driven Test Diseño



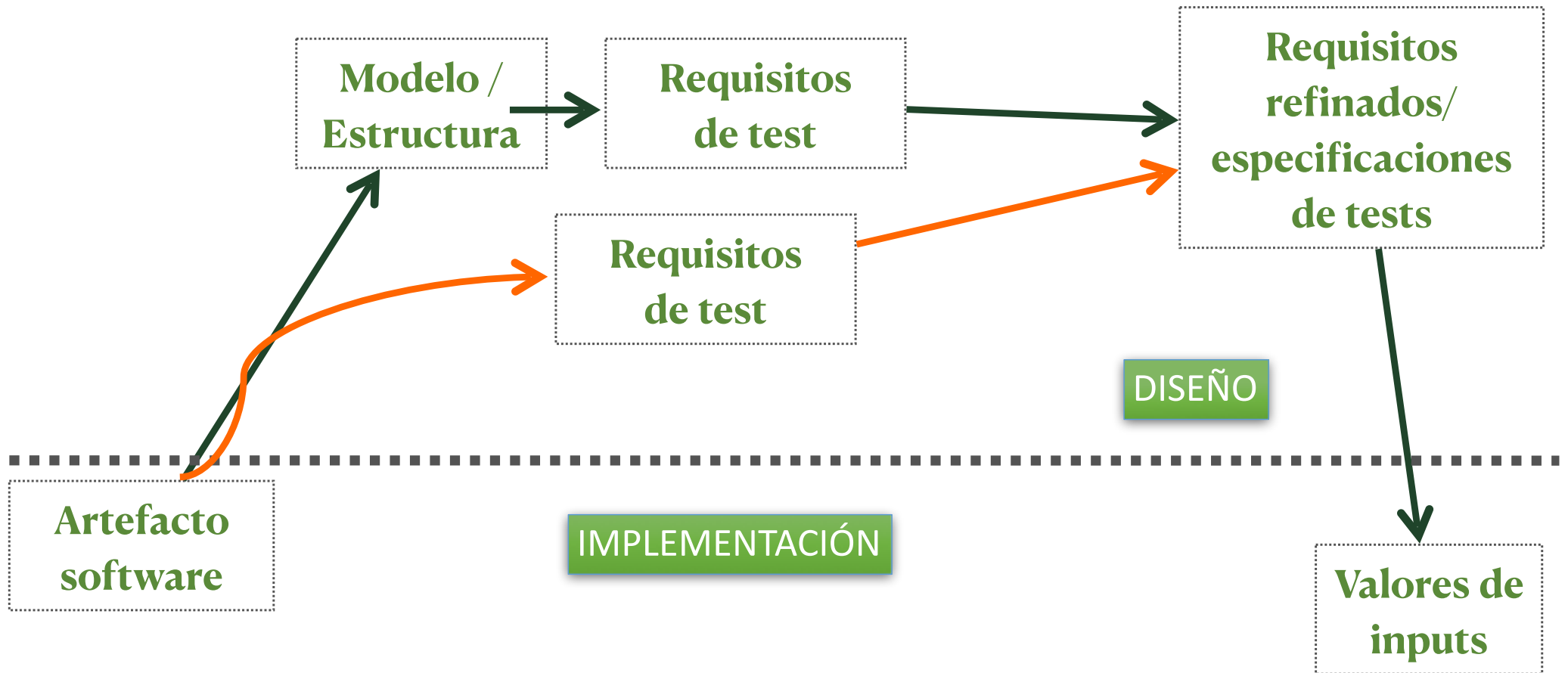
Model-Driven Test Diseño



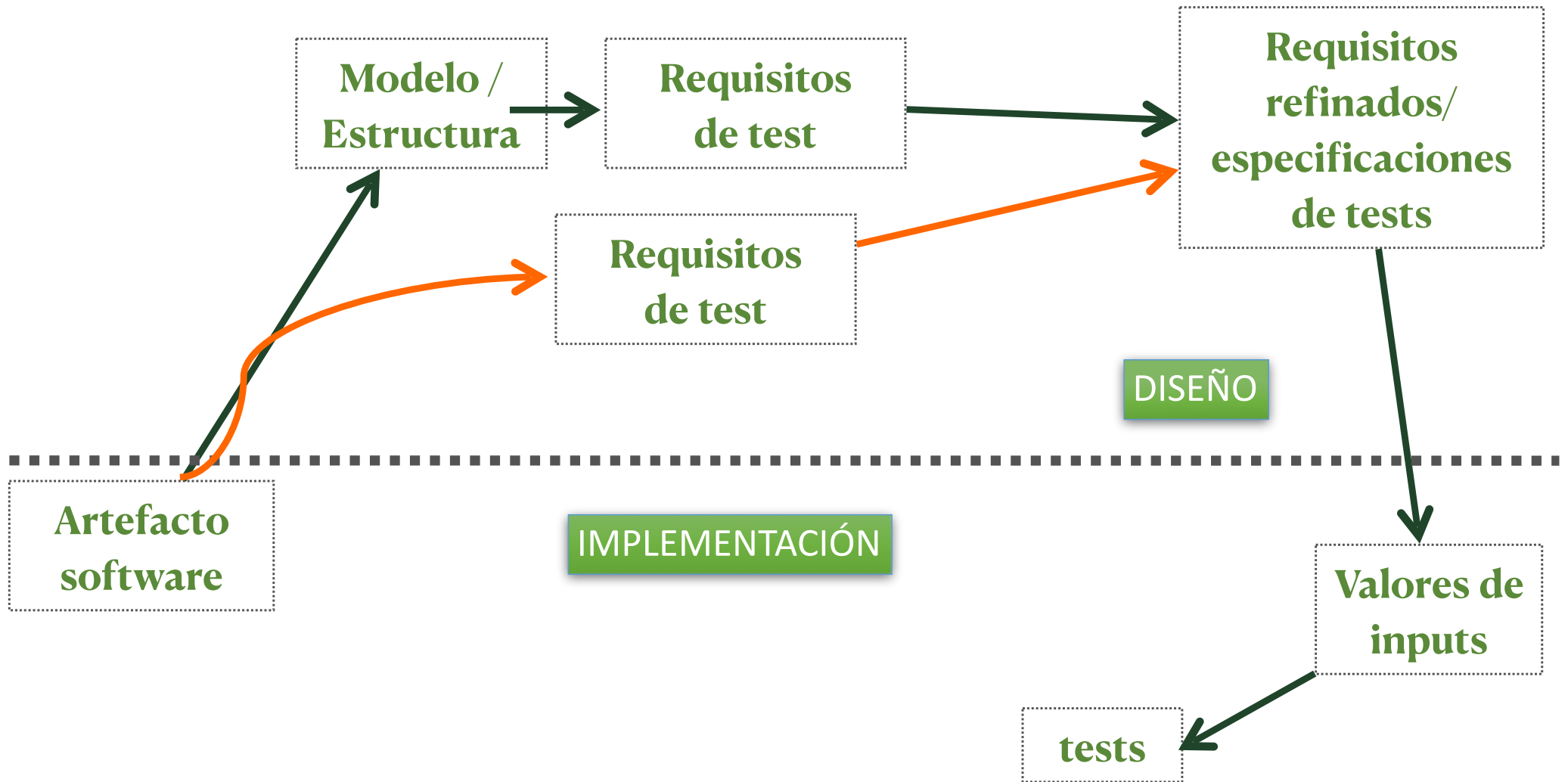
Model-Driven Test Diseño



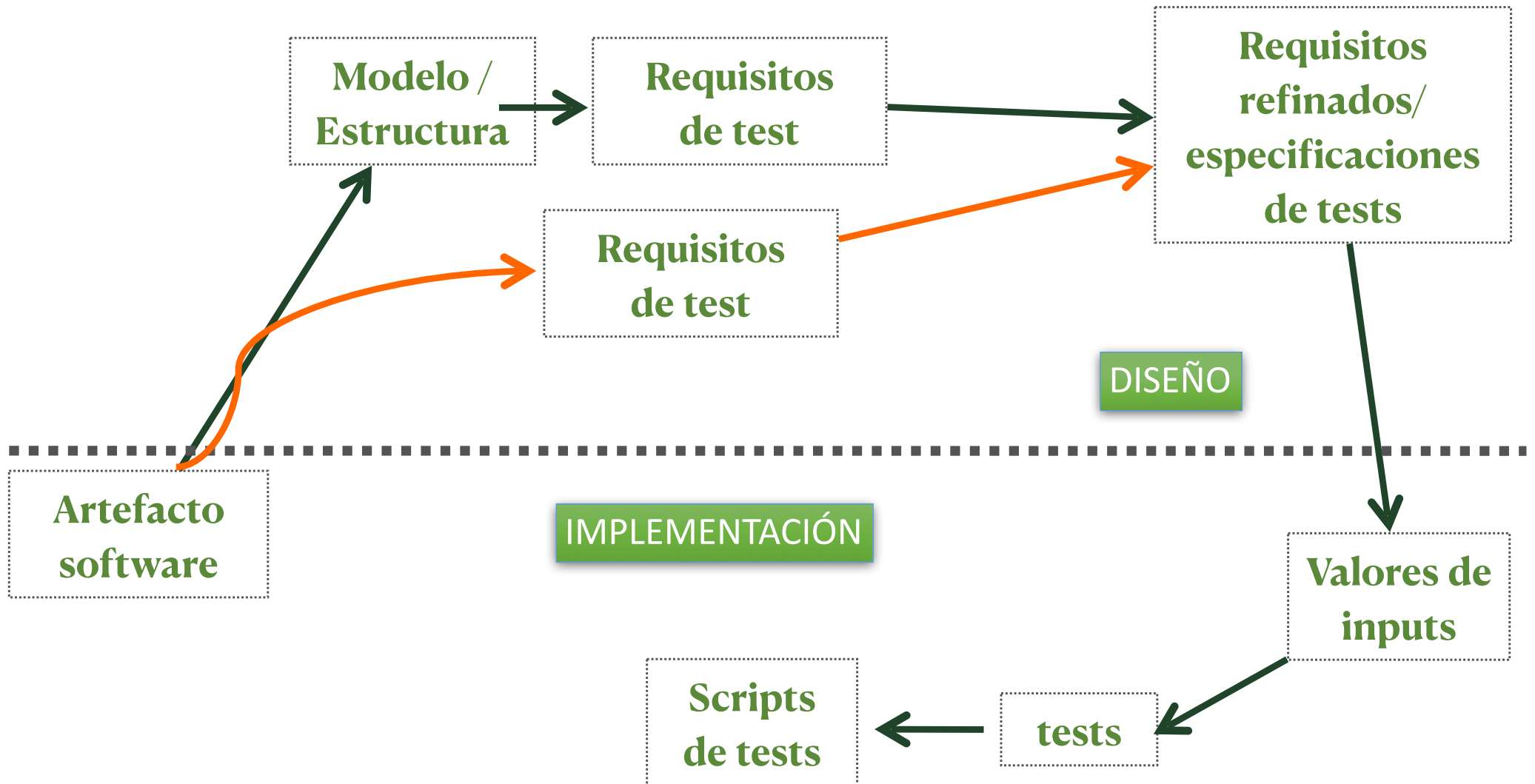
Model-Driven Test Diseño



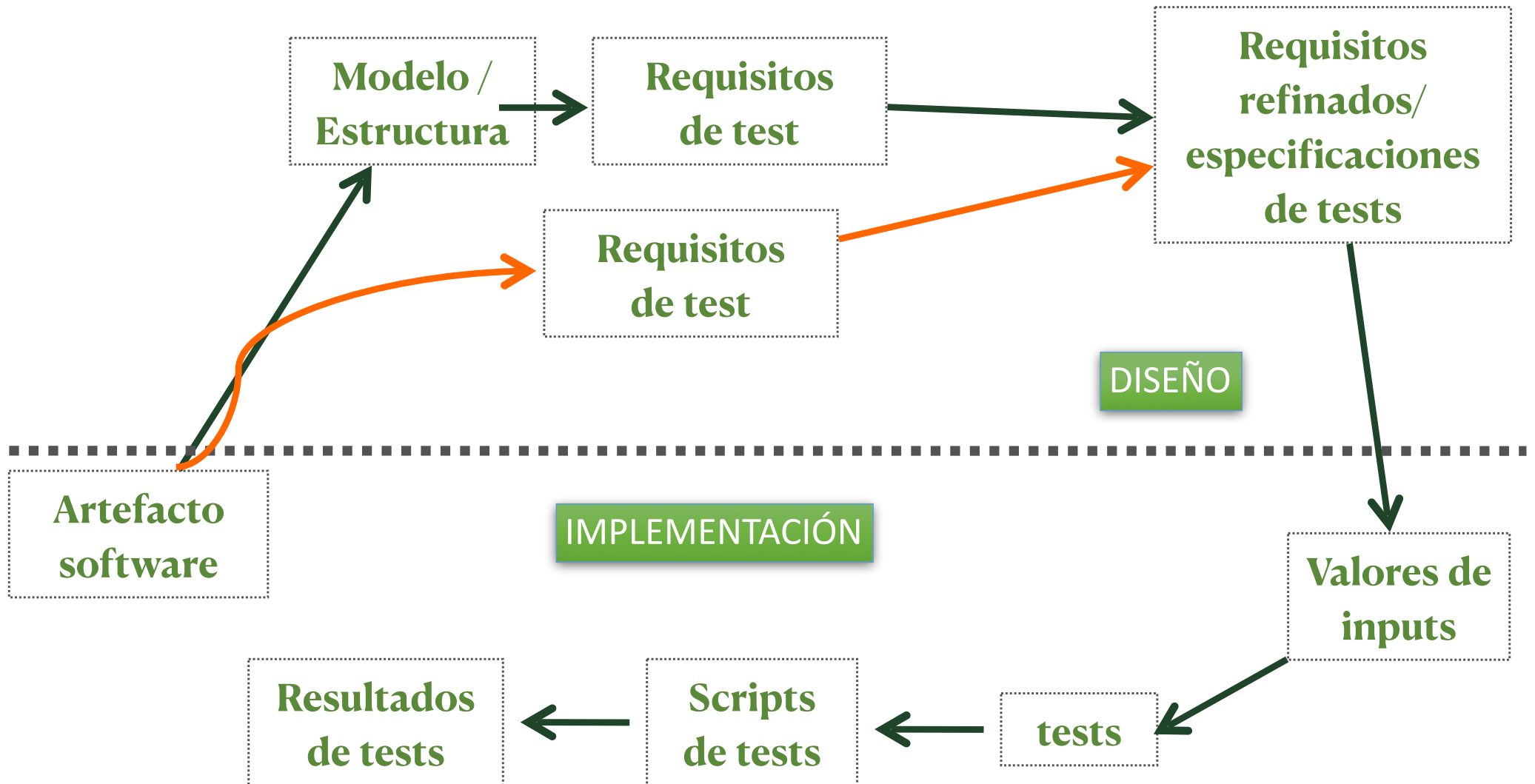
Model-Driven Test Diseño



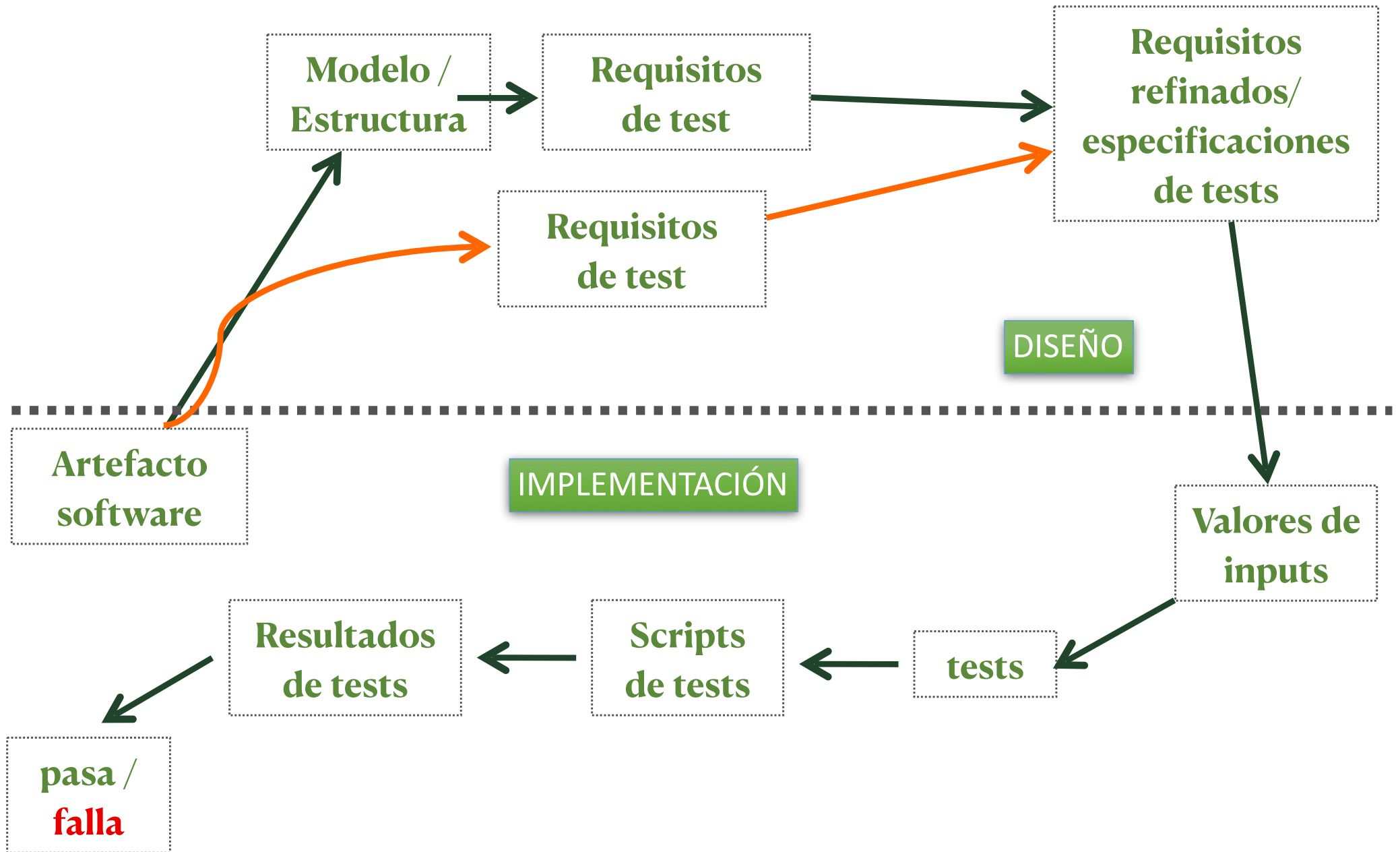
Model-Driven Test Diseño



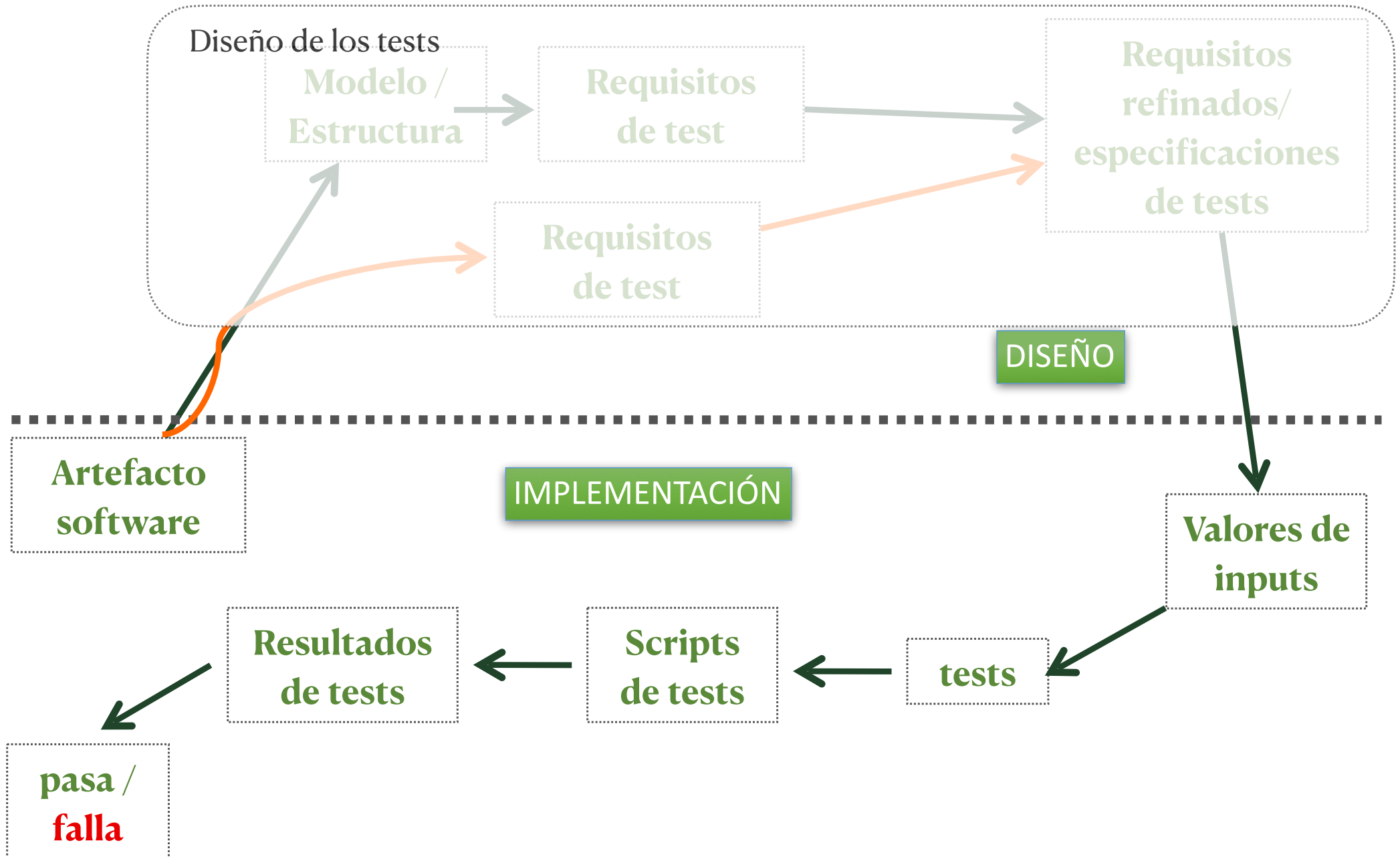
Model-Driven Test Diseño



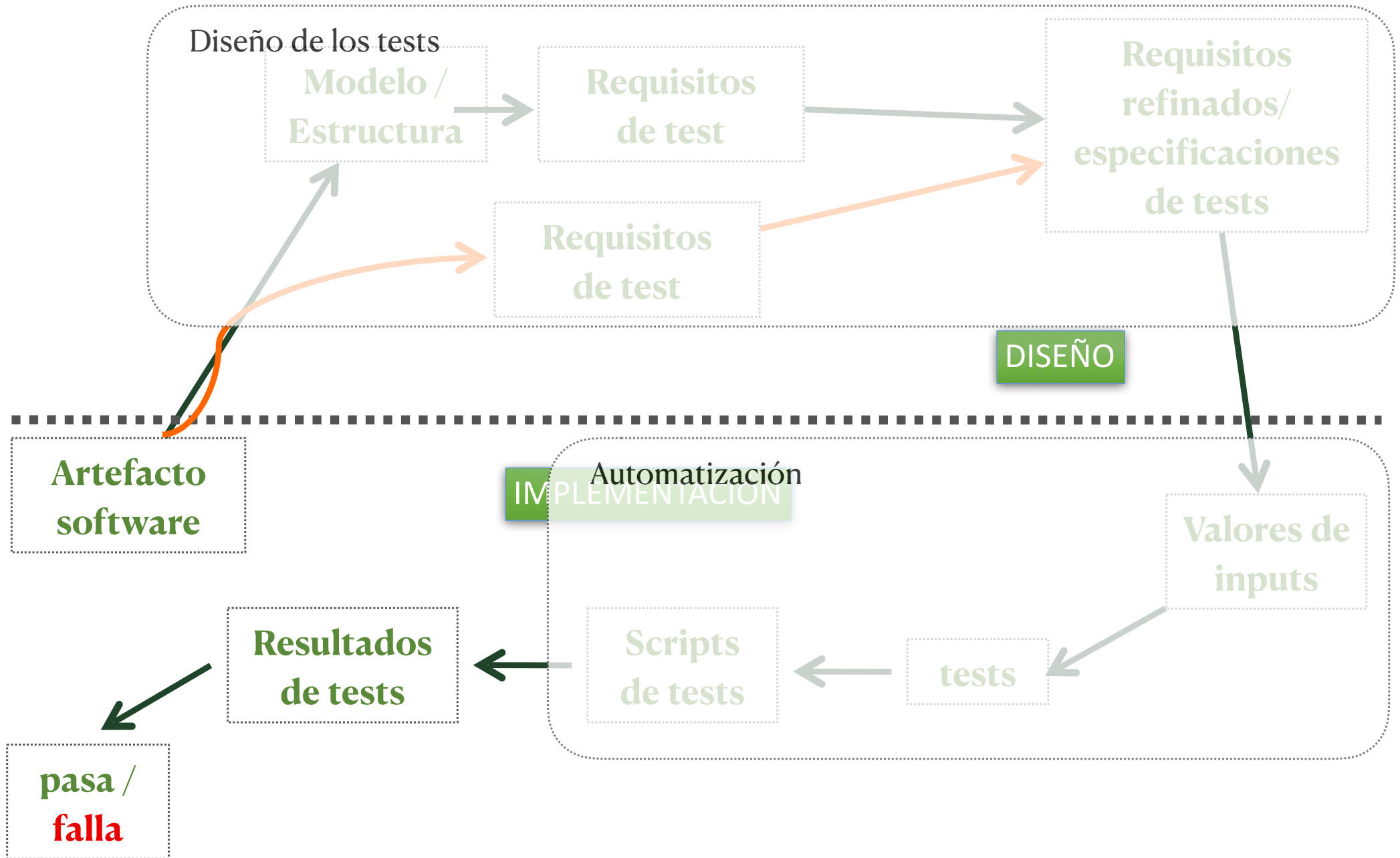
Model-Driven Test Diseño



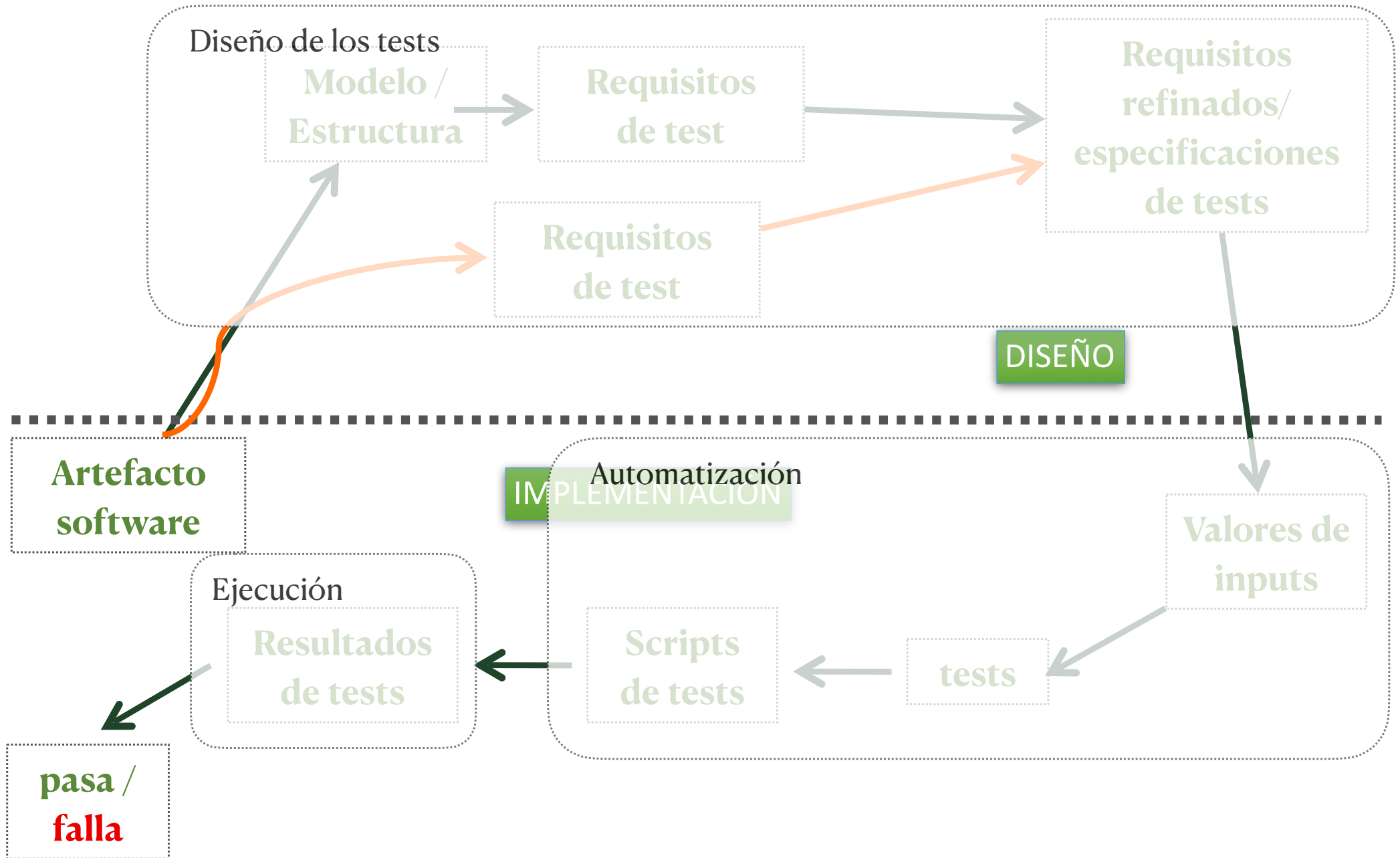
Model-Driven Test Diseño



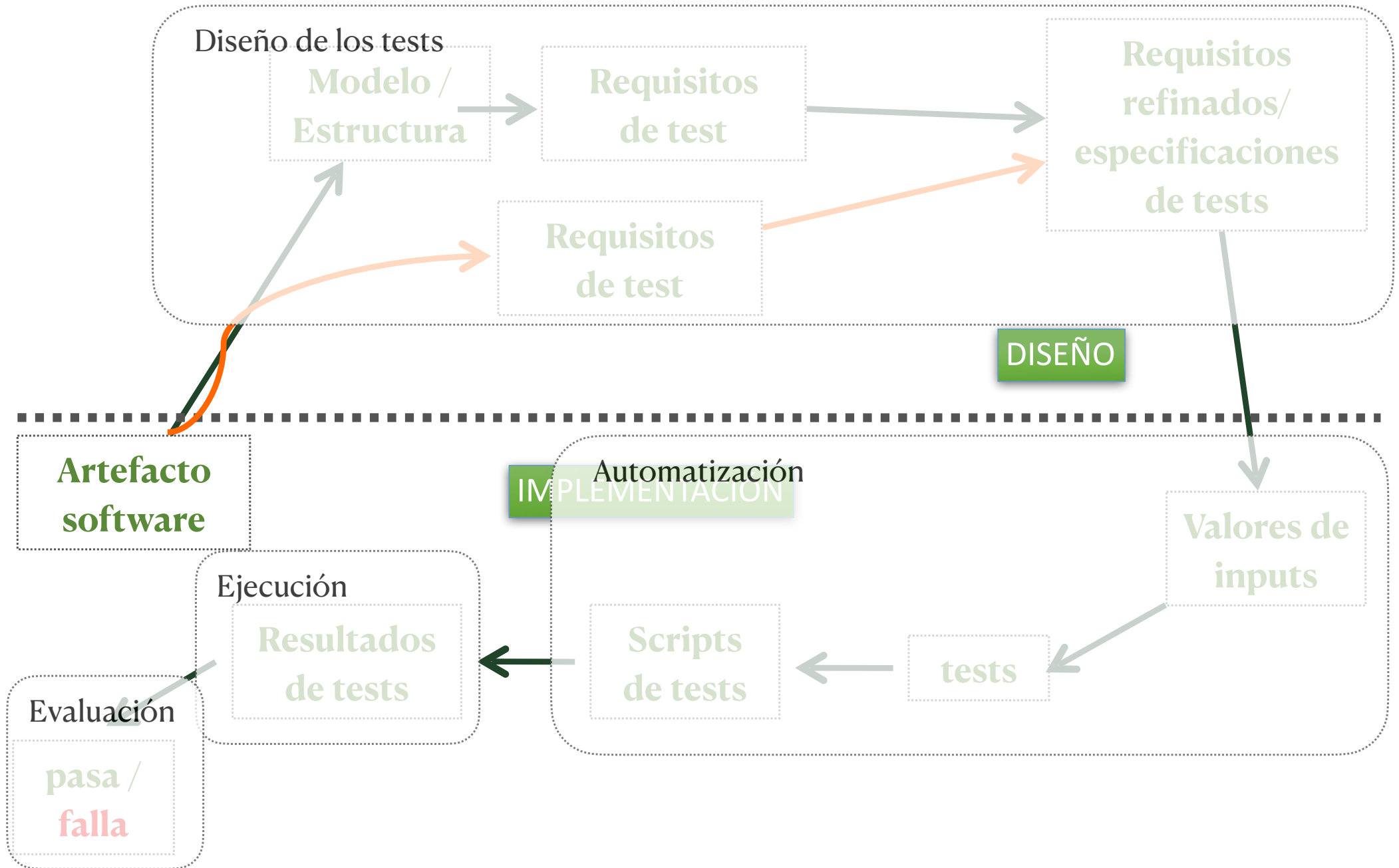
Model-Driven Test Diseño



Model-Driven Test Diseño



Model-Driven Test Diseño



Ejemplo

Ejemplo

Artefacto Software : Método Java

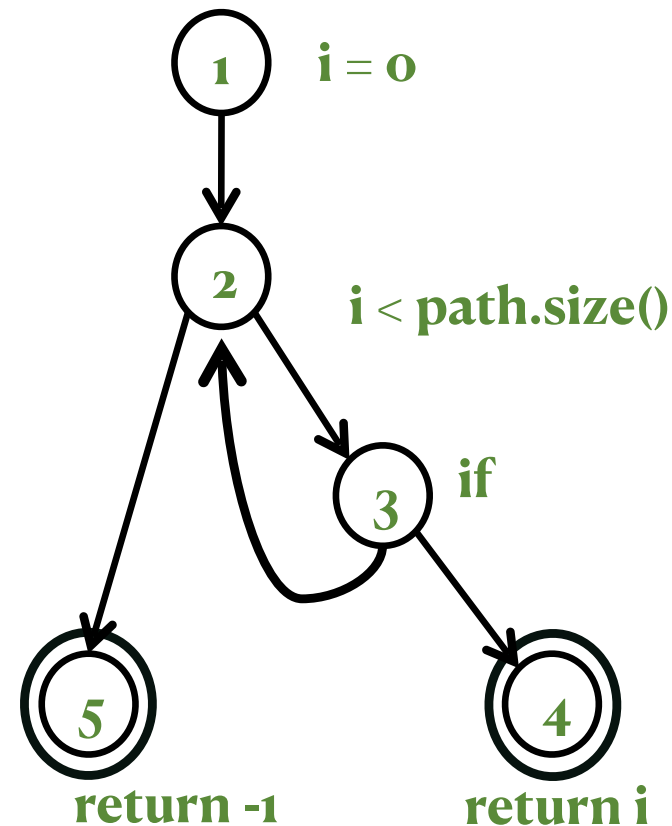
```
/**
 * Return index of node n at the
 * first position it appears,
 * -1 if it is not present
 */
public int indexOf (Node n)
{
    for (int i=0; i < path.size(); i++)
        if (path.get(i).equals(n))
            return i;
    return -1;
}
```

Ejemplo

Artefacto Software : Método Java

```
/**
 * Return index of node n at the
 * first position it appears,
 * -1 if it is not present
 */
public int indexOf (Node n)
{
    for (int i=0; i < path.size(); i++)
        if (path.get(i).equals(n))
            return i;
    return -1;
}
```

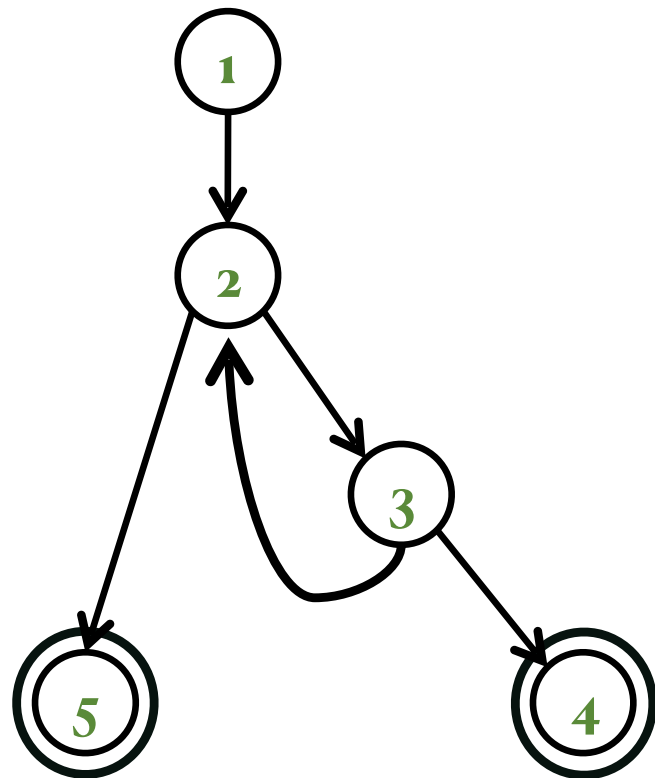
Modelo abstracto: Grafo de control de flujo



Ejemplo

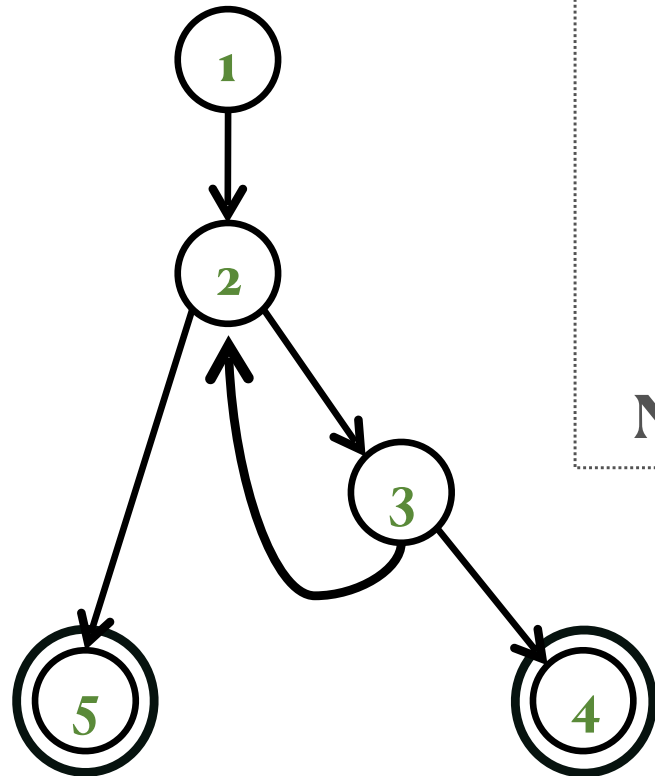
Ejemplo

Versión abstracta del
grafo



Ejemplo

Versión abstracta del
grafo



Aristas

1 2

2 3

3 2

3 4

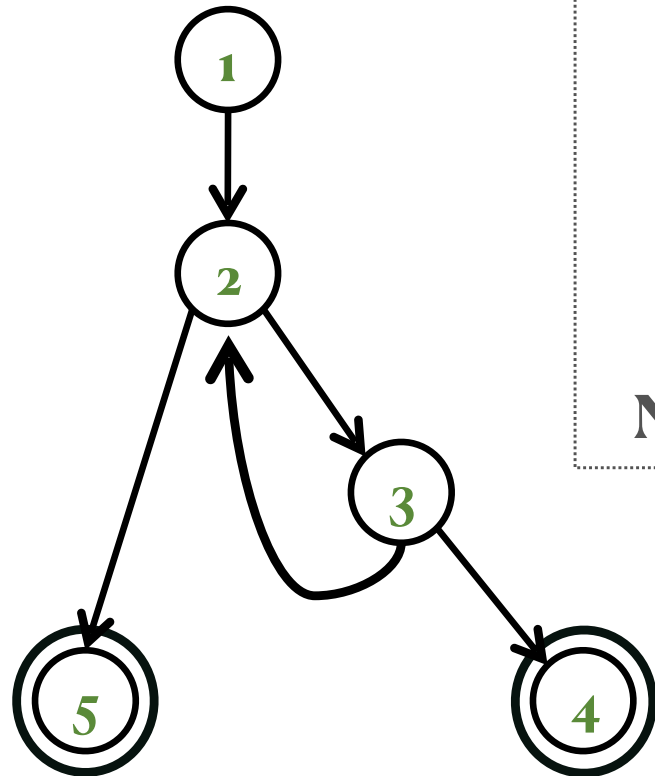
2 5

Nodo inicial: 1

Nodos finales: 4, 5

Ejemplo

Versión abstracta del
grafo



Aristas

1 2

2 3

3 2

3 4

2 5

Nodo inicial: 1

Nodos finales: 4, 5

Criterio: Cobertura
de pares de aristas

Requisitos:

1. [1, 2, 3]

2. [1, 2, 5]

3. [2, 3, 4]

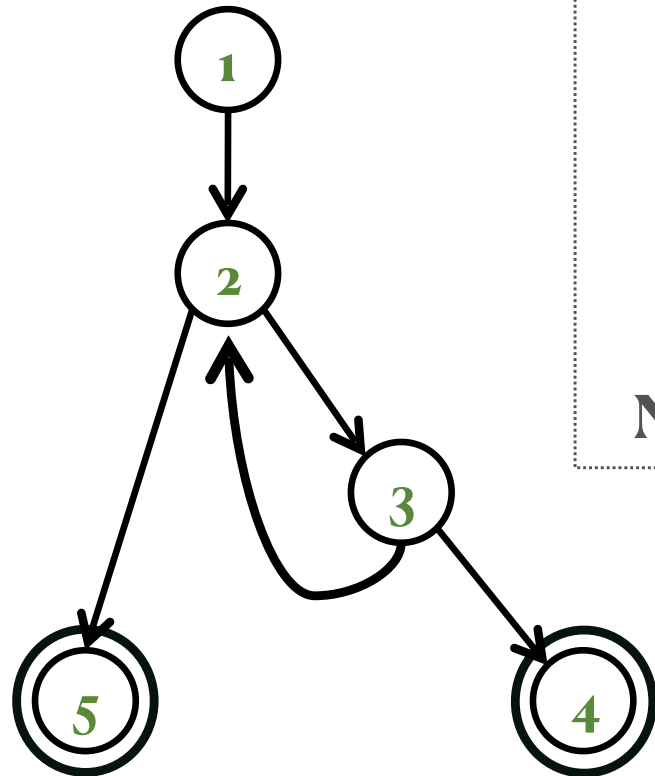
4. [2, 3, 2]

5. [3, 2, 3]

6. [3, 2, 5]

Ejemplo

Versión abstracta del
grafo



Aristas

1 2

2 3

3 2

3 4

2 5

Nodo inicial: 1

Nodos finales: 4, 5

Caminos de tests

[1, 2, 5]

[1, 2, 3, 2, 5]

[1, 2, 3, 2, 3, 4]

Criterio: Cobertura
de pares de aristas

Requisitos:

1. [1, 2, 3]

2. [1, 2, 5]

3. [2, 3, 4]

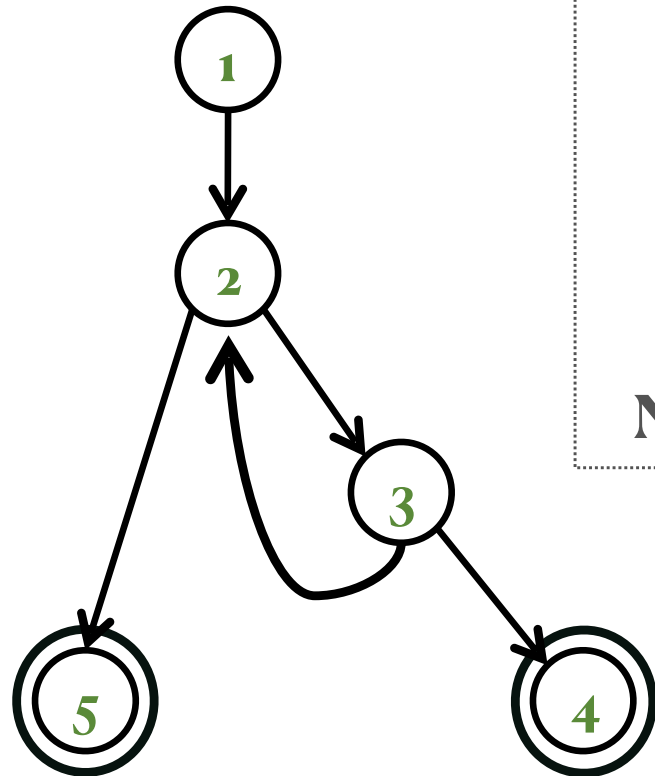
4. [2, 3, 2]

5. [3, 2, 3]

6. [3, 2, 5]

Ejemplo

Versión abstracta del
grafo



Aristas

1 2

2 3

3 2

3 4

2 5

Nodo inicial: 1

Nodos finales: 4, 5

Caminos de tests

[1, 2, 5]

[1, 2, 3, 2, 5]

[1, 2, 3, 2, 3, 4]

Criterio: Cobertura
de pares de aristas

Requisitos:

1. [1, 2, 3]

2. [1, 2, 5]

3. [2, 3, 4]

4. [2, 3, 2]

5. [3, 2, 3]

6. [3, 2, 5]

Encontrar valores...

Lectura recomendado

Leer el capítulo 2 del libro "Introduction to Software Testing" de Ammann y Offutt